In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')
```

In [ ]:
```
df = pd.read_csv("./Datasets/housing.csv")
df.head()
```

In [ ]:
```
df.shape
```

In [ ]:
```
df.isnull().sum()
```

In [ ]:
```
df.info()
```

In [ ]:
```
df.nunique()
```

In [ ]:
```
df.duplicated().sum()
```

In [ ]:
```
df['total_bedrooms'].median()
```

In [ ]:
```
df['total_bedrooms'].fillna(df['total_bedrooms'].median(), inplace=True)
```

In [ ]:
```
for i in df.iloc[:,2:7]:
    df[i] = df[i].astype('int')
```

In [ ]:
```
df.head()
```

In [ ]:
```
df.describe().T
```

In [ ]:
```
Numerical = df.select_dtypes(include=[np.number]).columns
Numerical
```

In [ ]:
```
for col in Numerical:
    plt.figure(figsize=(10,6))
    df[col].plot(kind='hist', bins=30,title=col, edgecolor="black")
    plt.ylabel("Frequency")
    plt.show()
```

In [ ]:
```
for col in Numerical:
    plt.figure(figsize=(6,6))
    sns.boxplot(df[col],color="blue")
    plt.title(col)
    plt.ylabel(col)
    plt.show()
```

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing

import warnings
warnings.filterwarnings('ignore')
```

In [ ]:
```
data = fetch_california_housing()

df = pd.DataFrame(data.data, columns=data.feature_names)
df['Target'] = data.target
```

In [ ]:
```
df.head()
```

In [ ]:
```
df.info()
```

In [ ]:
```
df.describe().T
```

In [ ]:
```
df.isnull().sum()
```

In [ ]:
```
plt.figure(figsize=(12,8))
df.hist(figsize=(12,8), bins=30, edgecolor="black")
plt.suptitle("Feature distributions", fontsize = 16)
plt.show()
```

In [ ]:
```
plt.figure(figsize=(12,8))
sns.boxplot(data=df)
plt.xticks(rotation=45)
plt.title("Boxplot of features to Identify Outliners")
plt.show()
```

In [ ]:
```
plt.figure(figsize=(12,8))
corr = df.corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Feature Correlation Matrix")
plt.show()
```

In [ ]:
```
sns.pairplot(df[["MedInc","AveRooms","HouseAge","Target"]], diag_kind="kd
e")
plt.show()
```

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

In [ ]:
```
data = load_iris()
X = data.data
y = data.target
```

In [ ]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [ ]:
```
cov_matrix = np.cov(X_scaled.T)
print(cov_matrix)
```

In [ ]:
```
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
print("Eigen Values:", eigenvalues)
print("Eigen Vectors:", eigenvectors)
```

In [ ]:
```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

In [ ]:
```
colors = ["red","green","blue"]
labels = data.target_names
```

In [ ]:
```
plt.figure(figsize=(8, 6))
for i in range(len(colors)):
    plt.scatter(X_pca[y == i, 0], X_pca[y == i, 1], color=colors[i], label=
labels[i])
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA on Iris Dataset (Dimensionality Reduction)')
plt.legend()
plt.grid()
plt.show()
```

In [ ]:
```
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
for i in range(len(colors)):
    ax.scatter(X_scaled[y == i, 0], X_scaled[y == i, 1], X_scaled[y == i,
2], color = colors[i], label = labels[i])
for i in range(3):
    ax.quiver(0, 0, 0, eigenvectors[i, 0], eigenvectors[i, 1], eigenvectors
[i, 2], color="black")
ax.set_xlabel('Sepal Length')
ax.set_ylabel('Sepal Width')
ax.set_zlabel('Petal Length')
ax.set_title('3D Data with Eigenvectors')
plt.legend()
plt.show()
```

In [ ]:
```
import pandas as pd
```

In [ ]:
```
df = pd.read_csv("./Datasets/training_data.csv")
df
```

In [ ]:
```
def find_s_algorithm(data):
    attributes = data.iloc[:,:-1].values
    target = data.iloc[:,-1].values
    for i in range(len(target)):
        if target[i] == "Yes":
            hypothesis = attributes[i].copy()
            break
    for i in range(len(target)):
        if target[i] == "Yes":
            for j in range(len(hypothesis)):
                if hypothesis[j] != attributes[i][j]:
                    hypothesis[j] = "?"
    return hypothesis
```

In [ ]:
```
print("Most Specific Hypothesis:",find_s_algorithm(df))
```

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import warnings

warnings.filterwarnings('ignore')
```

In [ ]:
```
np.random.seed(42)
values = np.random.rand(100)
```

In [ ]:
```
labels = ["Class 1" if x <= 0.5 else "Class 2" for x in values[:50]] + [None]*50
true_labels = ["Class 1" if x <= 0.5 else "Class 2" for x in values[50:]]
```

In [ ]:
```
print(labels)
```

In [ ]:
```
data = {
    "Point":[f"x{i+1}" for i in range(100)],
    "Values":values,
    "Labels":labels
}
```

In [ ]:
```
df = pd.DataFrame(data)
df.head()
```

In [ ]:
```
df.nunique()
```

In [ ]:
```
df.shape
```

In [ ]:
```
df.info()
```

In [ ]:
```
df.describe().T
```

In [ ]:
```
df.isnull().sum()
```

In [ ]:
```
num_col = df.select_dtypes(include=['int','float']).columns
num_col
```

In [ ]:
```
df[num_col].hist(bins=30, edgecolor="black")
plt.suptitle("Feature Distributions")
plt.show()
```

In [ ]:
```
labeled_df = df[df["Labels"].notna()]
X_train = labeled_df[["Values"]]
y_train = labeled_df["Labels"]
```

In [ ]:
```
unlabeled_df = df[df["Labels"].isna()]
X_test = unlabeled_df[["Values"]]
```

In [ ]:
```
k_values = [1,2,3,4,20,30]
accuracies = {}
```

In [ ]:
```
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    predictions = knn.predict(X_test)

    accuracy = accuracy_score(true_labels, predictions)*100
    accuracies[k] = accuracy
    print(f"Accuracy for {k}: {accuracy:.2f}")
    unlabeled_df[f"Label_{k}"] = predictions
```

In [ ]:
```
unlabeled_df.drop("Labels",inplace=True, axis=1)
```

In [ ]:
```
unlabeled_df
```

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

In [ ]:
```
def gaussian_kernel(x, x_query, tau):
    return np.exp(- (x - x_query) ** 2 / (2 * tau ** 2))

def locally_weighted_regression(X, y, x_query, tau):
    X_b = np.c_[np.ones(len(X)), X]
    x_query_b = np.array([1, x_query])
    W = np.diag(gaussian_kernel(X, x_query, tau))
    theta = np.linalg.pinv(X_b.T @ W @ X_b) @ X_b.T @ W @ y
    return x_query_b @ theta
```

In [ ]:
```
X1 = np.array([1, 2, 3, 4, 5])
y1 = np.array([1, 2, 1.3, 3.75, 2.25])
x_query1 = 3
tau1 = 1.0
y_pred1 = locally_weighted_regression(X1, y1, x_query1, tau1)
weights1 = gaussian_kernel(X1, x_query1, tau1)

plt.figure(figsize=(8, 5))
plt.scatter(X1, y1, color='blue', label='Data Points')
plt.scatter(x_query1, y_pred1, color='red', label=f'Prediction at x={x_quer
y1}')
for i in range(len(X1)):
    plt.plot([X1[i], X1[i]], [y1[i], y1[i] - weights1[i]], 'k-', lw=1)
    plt.scatter(X1[i], y1[i], s=weights1[i] * 200, color='green', alpha=0.
5)
plt.title("Graph 1: LWR with Weight Visualization")
plt.legend()
plt.grid()
plt.show()
```

In [ ]:
```
X2 = np.arange(1, 11)
y2 = np.array([1, 3, 2, 4, 3.5, 5, 6, 7, 6.5, 8])
X_query2 = np.linspace(1, 10, 100)
tau2 = 1.0
y_lwr2 = np.array([locally_weighted_regression(X2, y2, xq, tau2) for xq in
X_query2])

lr = LinearRegression()
lr.fit(X2.reshape(-1, 1), y2)
y_lin2 = lr.predict(X_query2.reshape(-1, 1))

plt.figure(figsize=(8, 5))
plt.scatter(X2, y2, color='blue', label='Data Points')
plt.plot(X_query2, y_lin2, 'k--', label='Linear Regression')
plt.plot(X_query2, y_lwr2, 'r', label=f'LWR (τ={tau2})')
plt.title("Graph 2: LWR vs Linear Regression")
plt.legend()
plt.grid()
plt.show()
```

In [ ]:
```
taus = [0.1, 0.5, 1.0, 5.0, 10.0]
colors = ['red', 'green', 'purple', 'orange', 'brown']

plt.figure(figsize=(10, 6))
plt.scatter(X2, y2, color='blue', label='Data Points')
plt.plot(X_query2, y_lin2, 'k--', label='Linear Regression')

for tau, color in zip(taus, colors):
    y_lwr = np.array([locally_weighted_regression(X2, y2, xq, tau) for xq i
n X_query2])
    plt.plot(X_query2, y_lwr, color=color, label=f'LWR (τ={tau})')

plt.title("Graph 3: Effect of τ in LWR")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.grid()
plt.show()
```

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score
import warnings
warnings.filterwarnings("ignore")
```

In [ ]:
```
df = pd.read_csv("./Datasets/Boston housing dataset.csv")
df.head()
```

In [ ]:
```
df.shape
```

In [ ]:
```
df.info()
```

In [ ]:
```
df.nunique()
```

In [ ]:
```
df.isnull().sum()
```

In [ ]:
```
df.duplicated().sum()
```

In [ ]:
```
df['CRIM'].fillna(df['CRIM'].mean(), inplace=True)
df['ZN'].fillna(df['ZN'].mean(), inplace=True)
df['CHAS'].fillna(df['CHAS'].mode()[0], inplace=True)
df['INDUS'].fillna(df['INDUS'].mean(), inplace=True)
df['AGE'].fillna(df['AGE'].median(), inplace=True)
df['LSTAT'].fillna(df['LSTAT'].median(), inplace=True)
```

In [ ]:
```
for i in df.columns:
    plt.figure(figsize=(6,3))
    plt.subplot(1, 2, 1)
    df[i].hist(bins=20, alpha=0.5, color='b',edgecolor='black')
    plt.title(f'Histogram of {i}')
    plt.xlabel(i)
    plt.ylabel('Frequency')

    plt.subplot(1, 2, 2)
    plt.boxplot(df[i], vert=False)
    plt.title(f'Boxplot of {i}')

    plt.show()
```

In [ ]:
```
corr = df.corr(method='pearson')
plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.xticks(rotation=90, ha='right')
plt.yticks(rotation=0)
plt.title("Correlation Matrix Heatmap")
plt.show()
```

In [ ]:
```
X = df.drop('MEDV', axis=1)
y = df['MEDV']
```

In [ ]:
```
scale = StandardScaler()
X_scaled = scale.fit_transform(X)
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=
0.2, random_state=42)
```

In [ ]:
```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\n--- Linear Regression (Boston Housing) ---")
print(f"MSE: {mean_squared_error(y_test, y_pred):.2f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.2f}")
print(f"R²: {r2_score(y_test, y_pred):.2f}")
```

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings("ignore")
```

In [ ]:
```
df = sns.load_dataset('mpg')
df.head()
```

In [ ]:
```
df.shape
```

In [ ]:
```
df.info()
```

In [ ]:
```
df.describe().T
```

In [ ]:
```
df.isnull().sum()
```

In [ ]:
```
df.duplicated().sum()
```

In [ ]:
```
df['horsepower'].fillna(df['horsepower'].median(), inplace=True)
```

In [ ]:
```
numerical = df.select_dtypes(include=['int','float']).columns
categorical = df.select_dtypes(include=['object']).columns
```

In [ ]:
```
for i in numerical:
    plt.figure(figsize=(10,4))
    plt.subplot(1, 2, 1)
    df[i].hist(bins=20, alpha=0.5, color='b',edgecolor='black')
    plt.title(f'Histogram of {i}')
    plt.xlabel(i)
    plt.ylabel('Frequency')

    plt.subplot(1, 2, 2)
    plt.boxplot(df[i], vert=False)
    plt.title(f'Boxplot of {i}')

    plt.show()
```

In [ ]:
```
plt.figure(figsize=(10, 8))
sns.heatmap(df[numerical].corr(), annot=True, cmap="coolwarm", fmt=".2f", l
inewidths=0.5)
plt.title("Auto MPG - Correlation Matrix")
plt.show()
```

In [ ]:
```
X = df[['horsepower']]
y = df['mpg']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ra
ndom_state=42)
```

In [ ]:
```
degree = 2
poly = PolynomialFeatures(degree)
X_poly_train = poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)

poly_model = LinearRegression()
poly_model.fit(X_poly_train, y_train)
y_pred = poly_model.predict(X_poly_test)
```

In [ ]:
```
plt.figure(figsize=(8, 5))
plt.scatter(X, y, alpha=0.5, label='Actual Data')
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
plt.plot(X_range, poly_model.predict(poly.transform(X_range)), color='red',
label='Polynomial Fit')
plt.xlabel('Horsepower')
plt.ylabel('MPG')
plt.title(f'Polynomial Regression (Degree {degree})')
plt.legend()
plt.grid(True)
plt.show()
```

In [ ]:
```
print("\n--- Polynomial Regression (Auto MPG) ---")
print(f"MSE: {mean_squared_error(y_test, y_pred):.2f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.2f}")
print(f"R²: {r2_score(y_test, y_pred):.2f}")
```

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusio
n_matrix

import math
import warnings
warnings.filterwarnings('ignore')
```

In [ ]:
```
data = pd.read_csv("./Datasets/Breast Cancer Dataset.csv")
data.head()
```

In [ ]:
```
data.shape
```

In [ ]:
```
data.info()
```

In [ ]:
```
data.isnull().sum()
```

In [ ]:
```
data.duplicated().sum()
```

In [ ]:
```
data.nunique()
```

In [ ]:
```
df = data.drop(['id'], axis=1)
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

X = df.drop('diagnosis', axis=1)
y = df['diagnosis']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ra
ndom_state=42)
```

In [ ]:
```
model = DecisionTreeClassifier(criterion='entropy', random_state=42)
model.fit(X_train, y_train)
```

In [ ]:
```
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred) * 100
classification_rep = classification_report(y_test, y_pred)

print("Accuracy: {:.2f}%".format(accuracy))
print("Classification Report:\n", classification_rep)
```

In [ ]:
```
plt.figure(figsize=(16, 10))
plot_tree(model, filled=True, feature_names=X.columns, class_names=['Benig
n', 'Malignant'])
plt.title("Decision Tree Visualization")
plt.show()
```

In [ ]:
```
def entropy(column):
    counts = column.value_counts()
    probabilities = counts / len(column)
    return -sum(probabilities * probabilities.apply(math.log2))

def conditional_entropy(data, feature, target):
    feature_values = data[feature].unique()
    weighted_entropy = 0
    for value in feature_values:
        subset = data[data[feature] == value]
        weighted_entropy += (len(subset) / len(data)) * entropy(subset[targ
et])
    return weighted_entropy

def information_gain(data, feature, target):
    total_entropy = entropy(data[target])
    feature_conditional_entropy = conditional_entropy(data, feature, targe
t)
    return total_entropy - feature_conditional_entropy
```

In [ ]:
```
print("\n--- Information Gain for each feature ---")
for feature in X.columns:
    ig = information_gain(df, feature, 'diagnosis')
    print(f"Information Gain for {feature}: {ig:.4f}")
```

In [ ]:
```
new_sample = np.array([[17.99, 10.38, 122.8, 1001.0, 0.1184, 0.2776, 0.300
1,
                        0.1471, 0.2419, 0.07871, 1.095, 0.9053, 8.589, 153.
4,
                        0.006399, 0.04904, 0.05373, 0.01587, 0.03003, 0.006
193,
                        25.38, 17.33, 184.6, 2019.0, 0.1622, 0.6656, 0.711
9,
                        0.2654, 0.4601, 0.1189]])

predicted_class = model.predict(new_sample)[0]
print("\nPredicted class for the new sample: ", "Malignant" if predicted_cl
ass == 1 else "Benign")
```

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB, MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix, classificatio
n_report, roc_auc_score
from sklearn.preprocessing import label_binarize
import warnings
warnings.filterwarnings("ignore")
```

In [ ]:
```
data = fetch_olivetti_faces()
X = data.data
Y = data.target
images = data.images
```

In [ ]:
```
print("Data Shape:", X.shape)
print("Target Shape:", Y.shape)
print("There are", len(np.unique(Y)), "unique persons in the dataset")
print("Size of each image is:", images.shape[1], "x", images.shape[2])
```

In [ ]:
```
def print_faces(images, target, top_n=36):
    top_n = min(top_n, len(images))
    grid_size = int(np.ceil(np.sqrt(top_n)))
    fig, axes = plt.subplots(grid_size, grid_size, figsize=(10, 10))
    fig.subplots_adjust(hspace=0.4)
    for i, ax in enumerate(axes.flat):
        if i < top_n:
            ax.imshow(images[i], cmap='gray')
            ax.set_title(f"Label: {target[i]}")
            ax.axis('off')
        else:
            ax.axis('off')
    plt.suptitle("Sample Faces from Olivetti Dataset", fontsize=16)
    plt.show()

print_faces(images, Y, top_n=36)
```

In [ ]:
```
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.3, ra
ndom_state=42)
```

In [ ]:
```
gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred_gnb = gnb.predict(X_test)
```

In [ ]:
```
gnb_acc = accuracy_score(y_test, y_pred_gnb)
print("\n[Gaussian Naive Bayes]")
print("Accuracy: {:.2f}%".format(gnb_acc * 100))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_gnb))
```

In [ ]:
```
nb = MultinomialNB()
nb.fit(X_train, y_train)
y_pred = nb.predict(X_test)
accuracy = round(accuracy_score(y_test, y_pred) * 100, 2)
print(f"Multinomial Naive Bayes Accuracy: {accuracy}%")
```

In [ ]:
```
misclassified_idx = np.where(y_pred_gnb != y_test)[0]
print(f"\nNumber of misclassified images: {len(misclassified_idx)}")
print(f"Accuracy: {round((1 - len(misclassified_idx)/len(y_test)) * 100,
2)}%")
```

In [ ]:
```
plt.figure(figsize=(10, 4))
for i in range(min(5, len(misclassified_idx))):
    idx = misclassified_idx[i]
    plt.subplot(1, 5, i + 1)
    plt.imshow(X_test[idx].reshape(64, 64), cmap='gray')
    plt.title(f"True:{y_test[idx]} Pred:{y_pred_gnb[idx]}")
    plt.axis('off')
plt.suptitle("Misclassified Faces (GaussianNB)", fontsize=14)
plt.show()
```

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
import warnings
warnings.filterwarnings('ignore')
```

In [ ]:
```
data = pd.read_csv("./Datasets/Wisconsin Breast Cancer dataset.csv")
data.head()
```

In [ ]:
```
data.shape
```

In [ ]:
```
data.info()
```

In [ ]:
```
data.isnull().sum()
```

In [ ]:
```
data.duplicated().sum()
```

In [ ]:
```
data.diagnosis.unique()
```

In [ ]:
```
df = data.drop(['id', 'Unnamed: 32'], axis=1)

df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

X = df.drop(columns=["diagnosis"])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

In [ ]:
```
explained_variance = pca.explained_variance_ratio_
print(f"PC1 variance: {explained_variance[0]:.4f}")
print(f"PC2 variance: {explained_variance[1]:.4f}")
print(f"Total variance explained: {np.sum(explained_variance):.4f}")
```

In [ ]:
```
wcss = []
K_range = range(1, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
```

In [ ]:
```
plt.figure(figsize=(8, 5))
plt.plot(K_range, wcss, marker='o')
plt.xlabel("Number of Clusters (k)")
plt.ylabel("WCSS (Inertia)")
plt.title("Elbow Method for Optimal k")
plt.grid(True)
plt.show()
```

In [ ]:
```
optimal_k = 2
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_pca)
```

In [ ]:
```
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap="viridis", alpha=0.
7)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s
=200, c='red', marker='X')
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clustering on PCA-Reduced Data")
plt.grid(True)
plt.show()
```